Identity and decision problem

Full name	Dominic Onen
Registration number	25/28259
Institutional username	ug2528259@ines.ac.rw
Verification code	**AI-A2-8259** (seed LLLL = 8259)
System	Musanze Harvest Intelligence Service: reproducible Python ML pipeline, saved models and FastAPI service
Repository	https://github.com/DominicOnen/AI_A2_2528259

Stakeholder and decision problem

The stakeholder is the **field officer of a potato cooperative in Musanze**, who works with member farmers. Farm records arrive in inconsistent spreadsheets and internet access can be unreliable. The officer has limited time and inputs and must decide **which farms to visit first** and **what harvest to plan for**. The service gives a quick, evidence-based second opinion that works offline and can later be connected to a mobile app.

Prediction questions

Task	Question the officer asks	Target
Regression	How many tons will this farm harvest, given rainfall, temperature, soil moisture, fertilizer, field size, season and cooperative support?	`yield_tons`
Classification	Is this farm at **high production risk** (yield per hectare below the median)?	`high_risk` (1 = high)
Clustering	Which farms have similar field conditions, so one visit plan can cover them?	none (unsupervised)

What the system returns

For one farm record the command-line tool and `POST /predict` return a yield estimate in tons, a risk prediction with its probability, a cluster number with a plain-language profile, and the verification code **AI-A2-8259**.

Ethical boundary

- The data are **synthetic**, generated from my registration digits. They do not describe real farms or people.
- The service is a **classroom decision-support prototype**. It does not replace an agronomist.
- It must **not** be used to decide whether a farmer gets finance, insurance or public services.
- A human must review every prediction before any action is taken.

How to read this report

Every number below is read from the generated files in `artifacts/` and `models/`, so the report agrees with the JSON and CSV outputs. Page 2 gives the data and reproducibility evidence, page 3 the pipeline design, page 4 regression and classification, page 5 clustering, page 6 the API and tests, and page 7 the model card and reflection.

Data and reproducibility

Data dictionary

Column	Generation rule	Use
`rainfall_mm`	Uniform 600 to 1400	Numeric feature
`avg_temp_c`	Uniform 12 to 26	Numeric feature
`soil_moisture_pct`	Uniform 20 to 80 (13 values deliberately missing)	Numeric feature
`fertilizer_kg`	Uniform 0 to 250 (13 values deliberately missing)	Numeric feature
`field_size_ha`	Uniform 0.2 to 4.0	Numeric feature
`season`	Random A or B	Categorical feature
`cooperative_support`	Random 0 or 1	Binary feature
`yield_tons`	0.004·rain + 0.06·soil + 0.02·fert + 1.30·size − 0.25·|temp − 18| + 0.70·support + season (+0.50 A, −0.20 B) + N(0, 0.80), minimum 0.50	Regression target
`high_risk`	1 when yield per hectare is below its median, else 0	Classification target

Personal seed rule and row count

- Seed LLLL = **8259** (last four digits of 25/28259); the only random source is `numpy.random.default_rng(8259)`.
- Base rows = 240 + (8259 mod 41) = 240 + 18 = **258**. Missing values per column: m = round(0.05 × 258) = **13**.
- Three exact duplicate rows are appended, giving **261** rows in `data/AI_A2_2528259.csv` (values saved with four decimals).

Missing and duplicate evidence

Check	Before cleaning	After cleaning	Handling
Rows	261	258	—
Exact duplicate rows	3	0	Dropped (first occurrence kept, original row ID retained)
Missing soil_moisture_pct	13	13	Median imputation inside the pipeline, fitted on training rows only
Missing fertilizer_kg	13	13	Same
Range violations	0 columns	0 columns	Run stops with an error if any value is out of range

Source: artifacts/validation_report.json. The CSV is never edited by hand; the generator injects these problems and the pipeline detects them.

Dataset fingerprint (SHA-256 of the CSV bytes)

`1c44802d7e37965c186e9aff1e98fc7919fe4a451fd3b68bd6101d14d89711dd`

Stored in artifacts/data_fingerprint.txt. Running `run_all.py` twice, and the automated test, give the same fingerprint.

Reproduce on a clean computer

```
python run_all.py --student-id ug2528259@ines.ac.rw --seed 8259
```

A seed that is not 8259 is rejected with an error before anything is written.

Pipeline design

Train/test separation

After cleaning there are 258 rows. One **stratified** split with `random_state = 8259` gives **206 training** and **52 test** rows (80/20). The same split is used for all three tasks, so no test row is seen by any fitting step. Stratifying on `high_risk` keeps both classes in the test set.

Preprocessing flow

CSV → validate schema, ranges, missing, duplicates → drop duplicates → split 206 / 52

ColumnTransformer (fitted on training rows only) → numbers: median impute + standard scale **+** season: one-hot **+** cooperative_support: pass through

model → 5-fold CV on training rows → choose model → score test rows once → save pipeline with joblib

The imputer, scaler and encoder are steps of a scikit-learn **Pipeline** together with the model. Training and inference therefore apply identical transformations, and `predict.py` and `app.py` load the saved pipelines.

Leakage controls

- Inputs are the seven features only. `yield_tons`, `high_risk` and yield per hectare are never inputs; `check_no_leakage` raises an error if one is added, and the API rejects extra fields.
- No imputer, encoder or scaler is fitted before the split. Model choice uses cross-validation inside the training rows; test rows are used only to report final scores.
- Clustering uses only the five scaled field-condition numbers. Yield and risk are reported afterwards, to describe clusters, never to form them.
- All random states are set to 8259, and models use one worker so results do not depend on thread timing.

Model candidates

Task	Baseline	Candidates	Selection rule
Regression	Mean of training yields	Linear regression, random forest (300 trees), gradient boosting	Simplest model within one standard error of the lowest 5-fold CV RMSE
Classification	Majority class	Logistic regression, random forest (300 trees)	Simplest model within one standard error of the highest 5-fold CV F1
Clustering	—	K-means with K = 2 to 6	Highest silhouette score on training rows

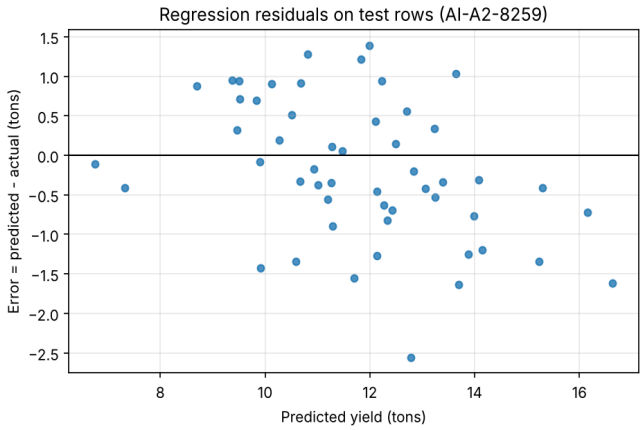
Why “simplest within one standard error”? With 206 training rows, cross-validation scores vary from fold to fold. If a simpler model is statistically as good as the best one, it is easier to explain, less likely to overfit and cheaper to serve. The rule uses training-row evidence only.

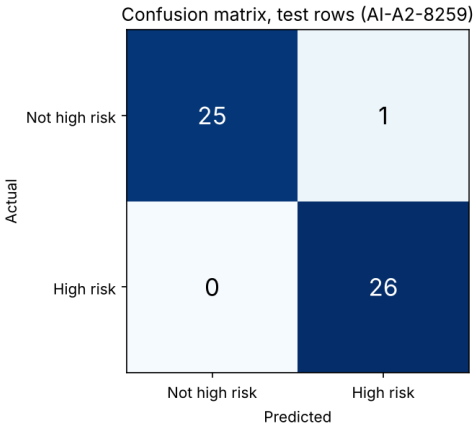
Regression and classification

Regression (52 test rows)	CV RMSE	Test MAE	Test RMSE	Test R²
mean baseline	2.316	2.017	2.535	-0.030
linear regression *(selected)*	1.021	0.756	0.912	0.867
random forest	1.318	1.121	1.414	0.680
gradient boosting	1.235	1.018	1.295	0.731

Classification (52 test rows)	CV F1	Precision	Recall	F1	ROC-AUC
majority baseline	0.262	0.000	0.000	0.000	0.500
logistic regression *(selected)*	0.933	0.963	1.000	0.981	0.991
random forest	0.943	0.857	0.923	0.889	0.969

Metric files: artifacts/regression_metrics.json and classification_metrics.json. The majority baseline predicts one class, so its ROC-AUC is 0.5 and its precision and recall are 0.





Selected models. linear regression for yield: test MAE 0.756 tons, RMSE 0.912, R² 0.867, against RMSE 2.535 for the mean baseline. **logistic regression** for risk: precision 0.963, recall 1.000, F1 0.981, ROC-AUC 0.991. Confusion matrix (actual × predicted): TN 25, FP 1, FN 0, TP 26. Random forest had a slightly higher CV F1 (0.943 against 0.933), but the gap is inside one standard error, so the simpler logistic regression was chosen by the rule.

Residuals. Errors are centred on zero, but lean positive for low predictions and negative for high ones: the model pulls extremes toward the middle. The true temperature effect is V-shaped, which a straight line cannot fit.

One important regression error (my interpretation)

For the farm in row 65 the model predicted 12.78 tons but the real yield was 15.33 tons, so it was 2.55 tons too low (the average error is 0.756 tons). The main reason is that this farm's soil moisture was missing in the data, so the model used the typical value of 48% while the true value was 67%. When the true value was used, the error fell to 1.44 tons. The rest is random variation in the harvest (+1.91 tons for this farm) that the model cannot know. Next time I would handle missing values more carefully or ask the officer to record soil moisture, because the estimate is only as good as its inputs. *Evidence: python -m src.error_analysis.*

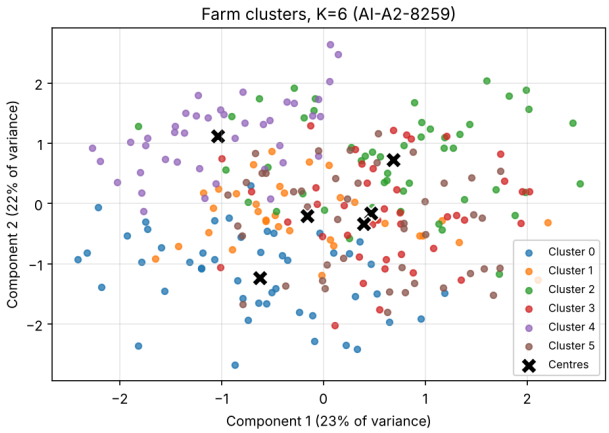
One important classification error (my interpretation)

For the farm in row 27 the model said high risk with a probability of 0.88, but the farm was not high risk. Its yield per hectare was 5.62, just above the median of 5.60, so it was almost exactly on the boundary. The model seems to rely a lot on field size, and this farm is large at 2.95 ha, like the high-risk farms (average 2.96 ha, against 1.13 ha for the others). This is a false alarm, and it is the only error among 52 test rows. A false alarm costs an officer an unnecessary visit, but a missed high-risk farm would be worse, so I would accept occasional false alarms and check borderline farms by hand.

Clustering

K	Silhouette	Choice
2	0.1566	
3	0.1624	
4	0.1739	
5	0.1804	
6	0.1842	selected (highest)

Source: artifacts/clustering_report.json. Features: rainfall_mm, avg_temp_c, soil_moisture_pct, fertilizer_kg, field_size_ha, scaled, with missing values imputed from training rows. Targets are not used.



Choice of K. K=6 has the highest silhouette score (0.184197) among K=2 to 6. The scores rise slowly with K and K = 6 is the edge of the tested range, so I treat the clusters as **practical visit groups**, not proven farm types. Silhouette scores are low because the synthetic farm conditions were drawn independently and uniformly, so there are no natural, well-separated groups. The clusters are convenient groupings for field visits, not distinct farm types.

Cluster profiles (training farms / test farms)

Cluster	Farms	Plain-language profile	High-risk rate*	Mean yield (t)*	Visit priority
0	36 / 12	Farms with high soil moisture, high fertilizer use, low rainfall.	44%	13.4	Average priority
1	27 / 7	Farms with low rainfall, low soil moisture, low field size.	26%	8.6	Lower priority
2	38 / 11	Farms with high temperature, high rainfall.	71%	12.2	Visit first
3	33 / 9	Farms with high field size, low soil moisture, low temperature.	91%	12.4	Visit first
4	33 / 8	Farms with low field size, high rainfall.	0%	10.9	Lower priority
5	39 / 5	Farms with low fertilizer use, high soil moisture.	59%	11.1	Average priority

*Reported after the clusters were formed, only to interpret them. Overall high-risk rate in the training rows: 50%.

One farm cluster explained (my interpretation)

Cluster 3 contains 33 training farms with large fields (about 3.1 ha), low soil moisture (32%) and low temperature (16 °C). Afterwards, 91% of these farms were high risk, compared with 50% for all farms. This means the cooperative's officer should visit this group first. However, the clusters overlap quite a lot (silhouette about 0.18), so I treat them as a rough guide to prioritise visits and not as exact categories.

Operational meaning. The field officer should visit the Cluster 3 group first, since most of its farms are at risk. Clusters with small fields and a very low risk rate (see table) can wait. Because the silhouette score is low (about 0.18), a farm near a cluster boundary should be judged on its own record and not only on its cluster label.

API and testing

Endpoint	Purpose	Success / failure
`GET /health`	Is the service up and are the models loaded?	200 `{"status": "ok", "verification_code": "AI-A2-8259"}`
`GET /model-info`	Selected models, dataset fingerprint, seed, synthetic-data flag, verification code	200; 503 if models are missing
`POST /predict`	Typed farm record in; yield, risk, probability, cluster, code out	200; **422** for invalid input

Valid request → 200

```
{"rainfall_mm": 1000.0, "avg_temp_c": 18.5,
"soil_moisture_pct": 55.0, "fertilizer_kg": 120.0,
"field_size_ha": 1.8, "season": "A",
"cooperative_support": 1}
```

```
{
 "yield_estimate_tons": 12.3615,
 "risk_prediction": "not_high",
 "high_risk_probability": 0.207,
 "cluster": 5,
 "verification_code": "AI-A2-8259"
}
```

Invalid request → 422

```
{"rainfall_mm": 2500.0, "avg_temp_c": "warm",
"soil_moisture_pct": 55.0, "fertilizer_kg": 120.0,
"season": "C", "cooperative_support": 1}
```

```
{"error": "invalid_input",
 "problems": [
  {"field": "rainfall_mm", "message": "Input
should be less than or equal to 1400"},
  {"field": "avg_temp_c", "message": "Input should
be a valid number"},
  {"field": "field_size_ha", "message": "Field
required"},
  {"field": "season", "message": "Input should be
'A' or 'B'"}
 ]}
```

Failure behaviour

- Strict typing: a string such as "warm" is not accepted for a number; `cooperative_support` must be 0 or 1; `season` must be A or B; extra fields (for example a target) are rejected.
- Errors list each bad field and a short message only: no stack trace, path or model internals, and the bad values are not echoed.
- The command-line tool exits with code 2 for invalid input and 1 if the models are missing, with a message to run `run_all.py`.

Test summary: 17 tests, all passing (pytest -q)

Group	Tests	What is checked
Valid inference	1	all fields, code, ranges
Invalid input: missing, wrong type, out of range, bad season / extra field	4	rejected, field named
Reproducibility: same fingerprint; pipeline twice, same metrics	2	bytes and JSON metrics
Data problems: 13+13 missing, 3 duplicates, leakage guard, schema	3	checks fail loudly
Identity (wrong seed rejected)	1	seed must be 8259
Command line: valid record; invalid record exits with code 2	2	failure path
API: valid /predict, 422 with fields, non-JSON body, /health and /model-info	4	typed errors, code

Development environment (Linux, Python 3.13): 17 passed. Full list: evidence/AI_A2_2528259_TEST_LOG.pdf. Most tests exercise failure paths, not only the happy path.

Model card and reflection

Model card

Item	Detail
Model	Linear regression (yield), logistic regression (risk), K-means with K = 6 (field-condition groups), all in scikit-learn pipelines. Trained on 206 synthetic rows.
Intended use	Classroom demonstration of a decision-support tool: helping a field officer prioritise visits and plan an approximate harvest.
Prohibited use	Deciding access to finance, insurance or public services; replacing an agronomist; use on real farms without retraining and validation on real data; ranking or penalising individual farmers.
Synthetic-data status	All data are generated from my registration digits (8259). Performance on this data says nothing certain about real farms.
Human oversight	A person reviews every prediction and can override it. Probabilities near 0.5 and farms near a cluster boundary should be treated as uncertain and checked in the field.

Limitations

- Synthetic, independent, uniform inputs: no real-world messiness, and clusters are weak (silhouette about 0.18).
- Only 52 test rows, so metrics would move noticeably with another split. The high recall (1.000) is not a guarantee.
- The linear model cannot represent the V-shaped temperature effect, and median imputation adds error where soil moisture or fertilizer is missing (see the error on page 4).
- Inputs outside the training ranges are rejected, so the service cannot say anything about unusual farms.

Risks

- **False reassurance:** a high-risk farm predicted as safe could be skipped. Mitigation: show the probability, keep the human review step.
- **Over-trust:** a confident number can look more precise than it is. Mitigation: show the disclaimer in every response.
- **With real data:** bias against small or remote farms if they are under-represented, and privacy of farm records.

What I would improve with real data

- Check units and collection methods with the cooperative, and agree with field staff how to treat missing values instead of silently using the median.
- Add a temperature-distance feature or a non-linear model, and compare models on repeated cross-validation with more rows.
- Calibrate the probabilities, report prediction intervals for yield, and test accuracy separately for small and large farms.
- Monitor the model each season for drift and retrain with new harvest records.

Reflection

The main thing I learned is that the best-looking model is not always the right choice. Random forest scored slightly higher on cross-validation for risk, but the difference was within noise, so the simpler logistic regression is the more honest and explainable choice. I also learned that the target “high risk” is mostly a field-size effect, which only became clear after looking at errors and clusters instead of accuracy alone.